

10-16-25 Project Overview

Approaching object tracking in radar

J.C. Vaught

Undergraduate Assistants: Samuel Cancilla and Nathan Kirk
Department of Mechanical Engineering

Big Picture

Many AI models exist to detect objects, but few of them are trained for small maritime objects.

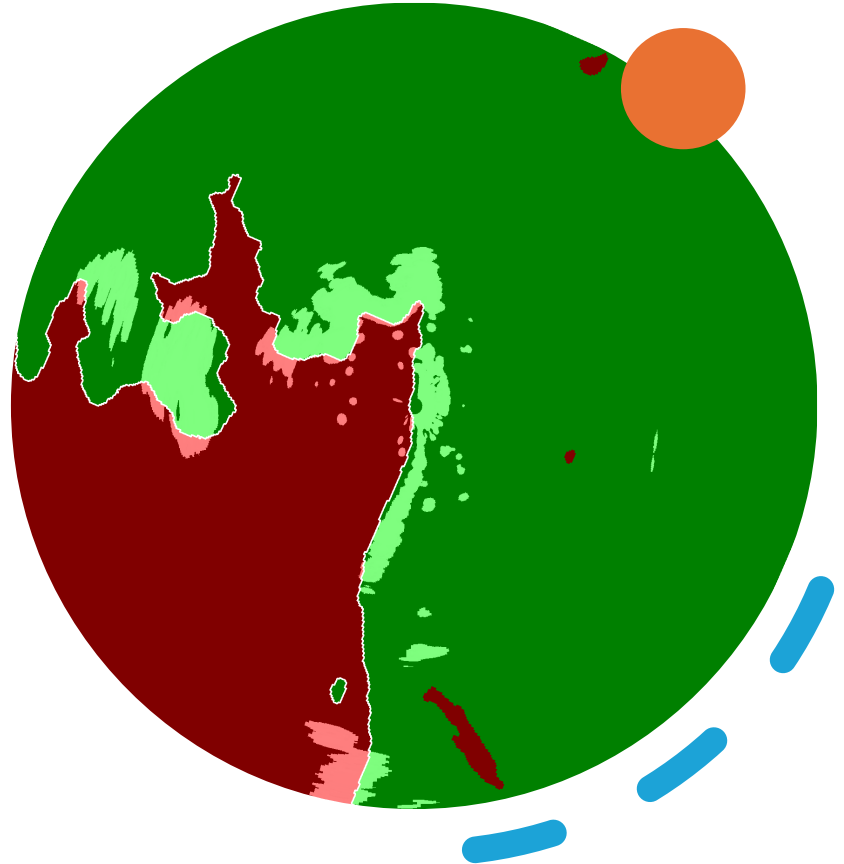
Training AI to identify marine objects requires labeling thousands of images, a tedious and costly process



What does radar do for us?

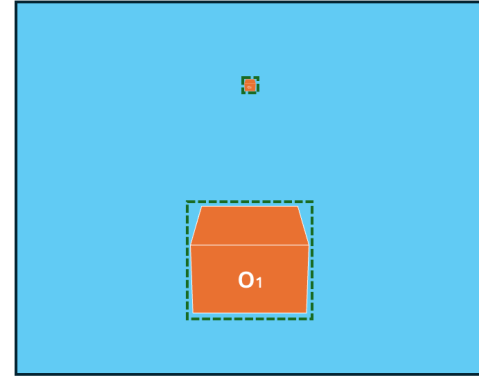
Main Goal: To create a system that automatically finds objects in camera images, making the data labeling process faster and more efficient.

The Core Idea: We will use one sensor (Radar) to "cue" another sensor (a Camera), telling it (or a person) where to look for objects.

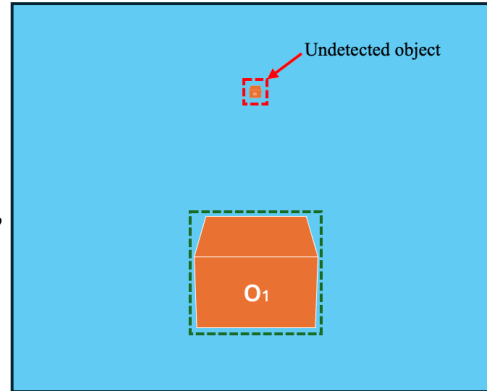


Our Tools

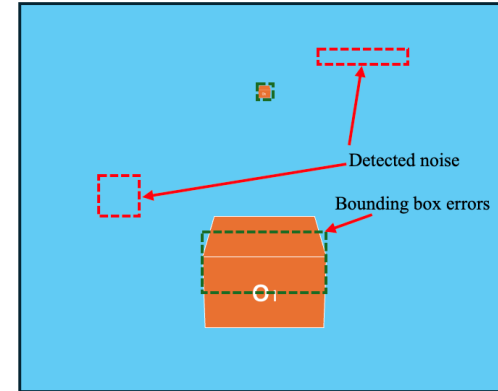
- Camera
 - Gives us a high-resolution, color picture of the world. Great for identifying what an object is (a boat, a buoy, etc.).
 - Weakness: Doesn't work well in fog, rain, or at night.
- Radar
 - Sends out radio waves and listens for echoes. Excellent at detecting objects at long distances, day or night, in any weather.
 - Weakness: Gives a low-resolution, "blob-like" picture. Can't easily tell a boat from a pylon.



(c) Sensor Fusion

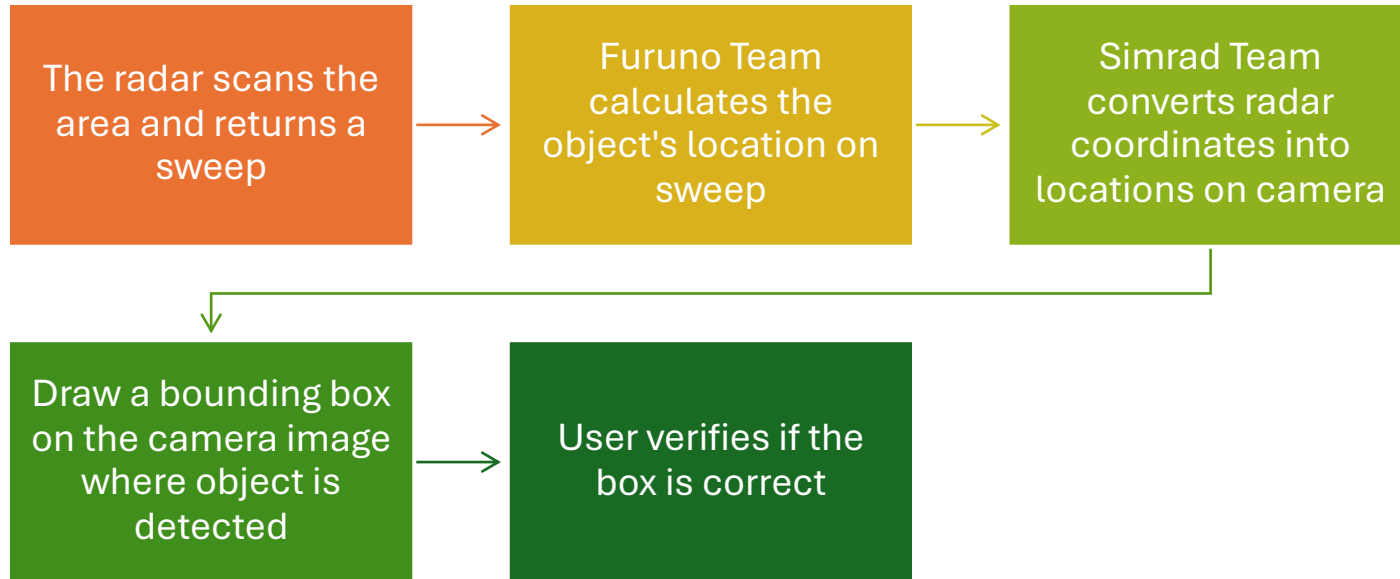


(a) Vision only



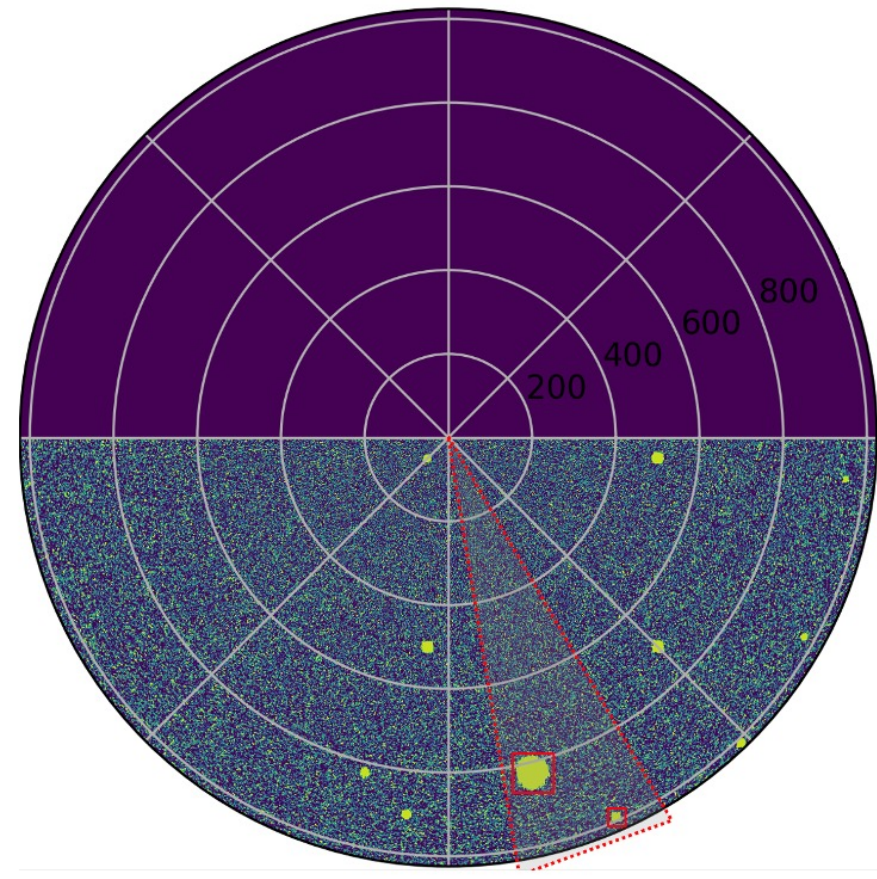
(b) Radar only

Auto-labelling



Method 2

(a) Radar Object Detection



(b) Radar Overlay for Annotator



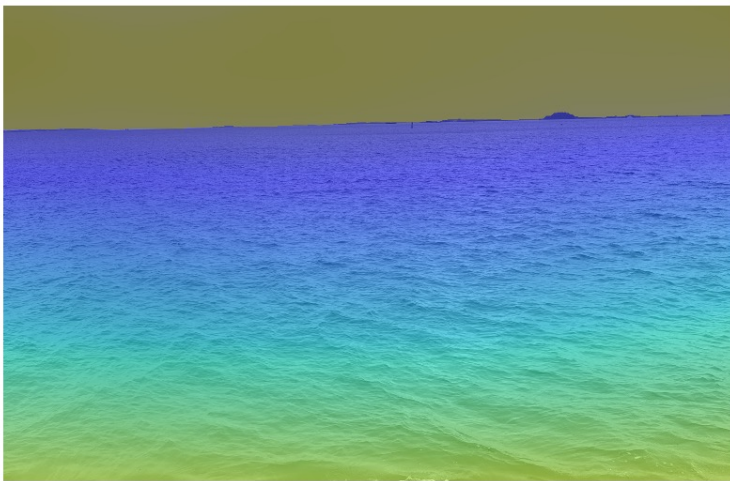
Method 1



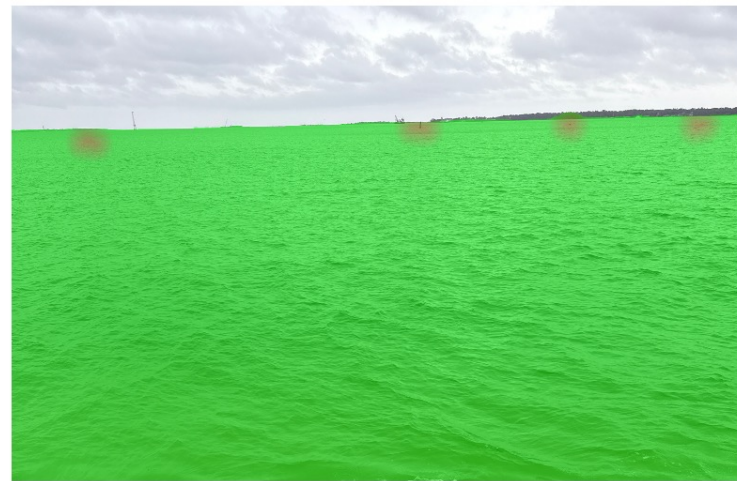
(a) Original image



(b) Segmentation



(c) Depth Map



(d) Radar overlay

The Furuno Team



(Furuno Team uses Furuno Radar, 24")

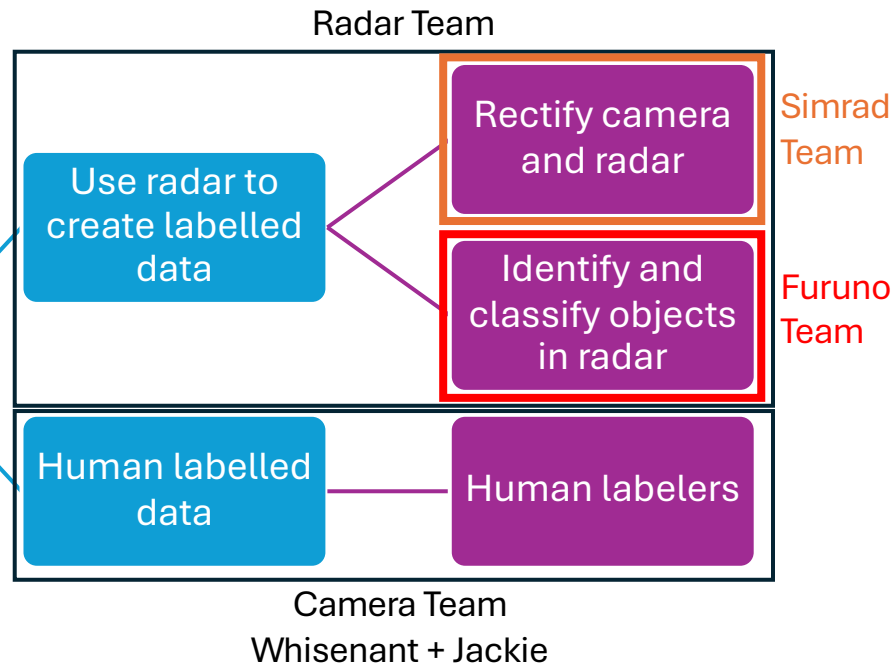
Goal

Develop model
to identify small
objects in ocean

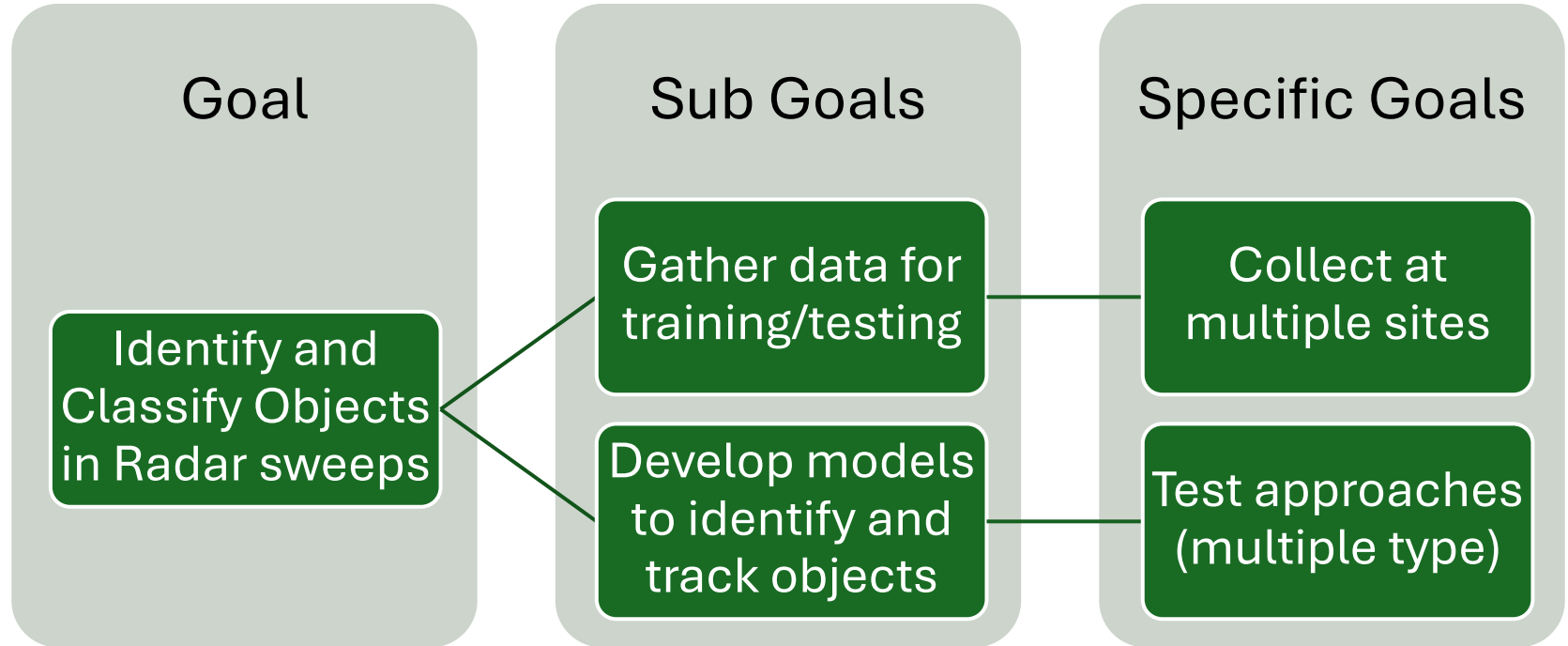
Sub Goal

Gather labelled
data for model
training

Overview of Project

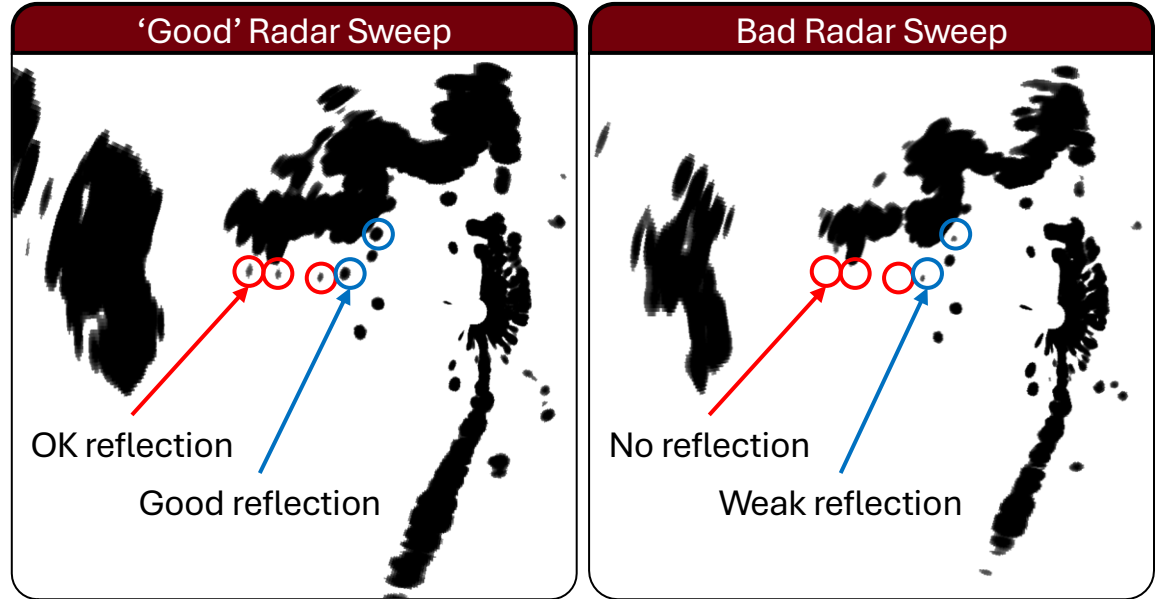


Overview of Furuno Team

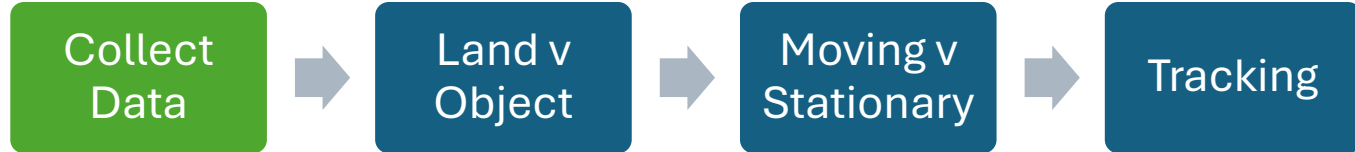


The Radar Problem

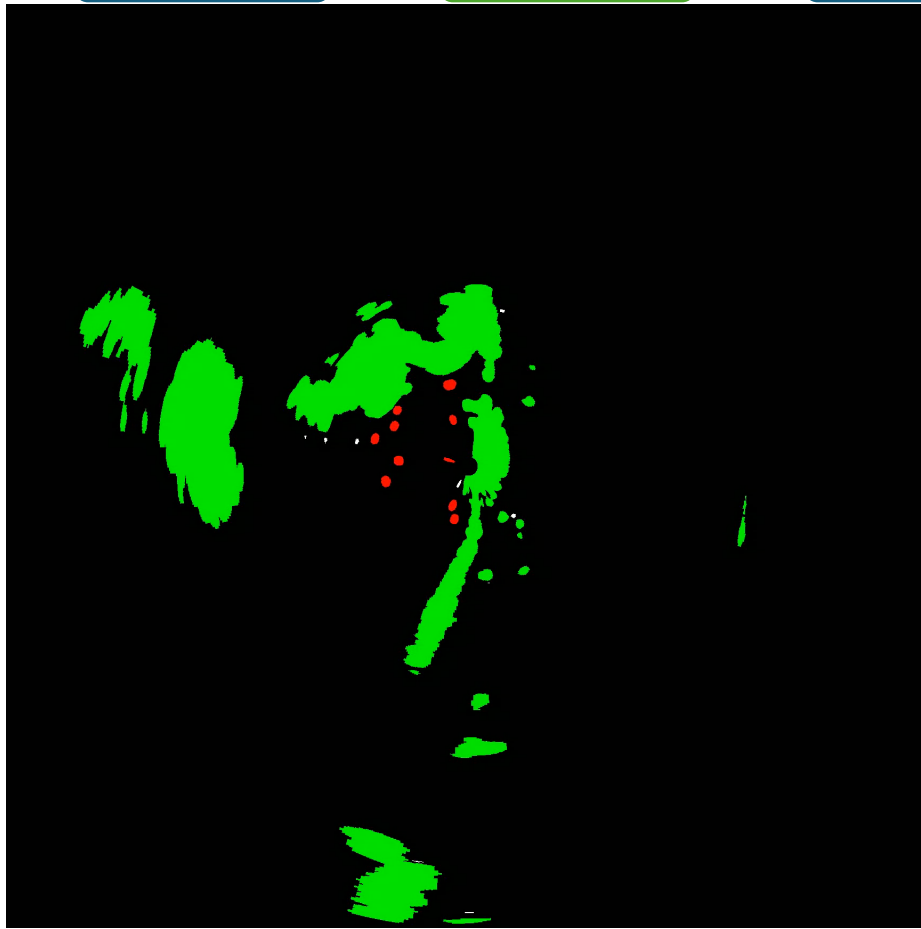
- Oftentimes the sweeps will not capture every object due to a variety of issues.



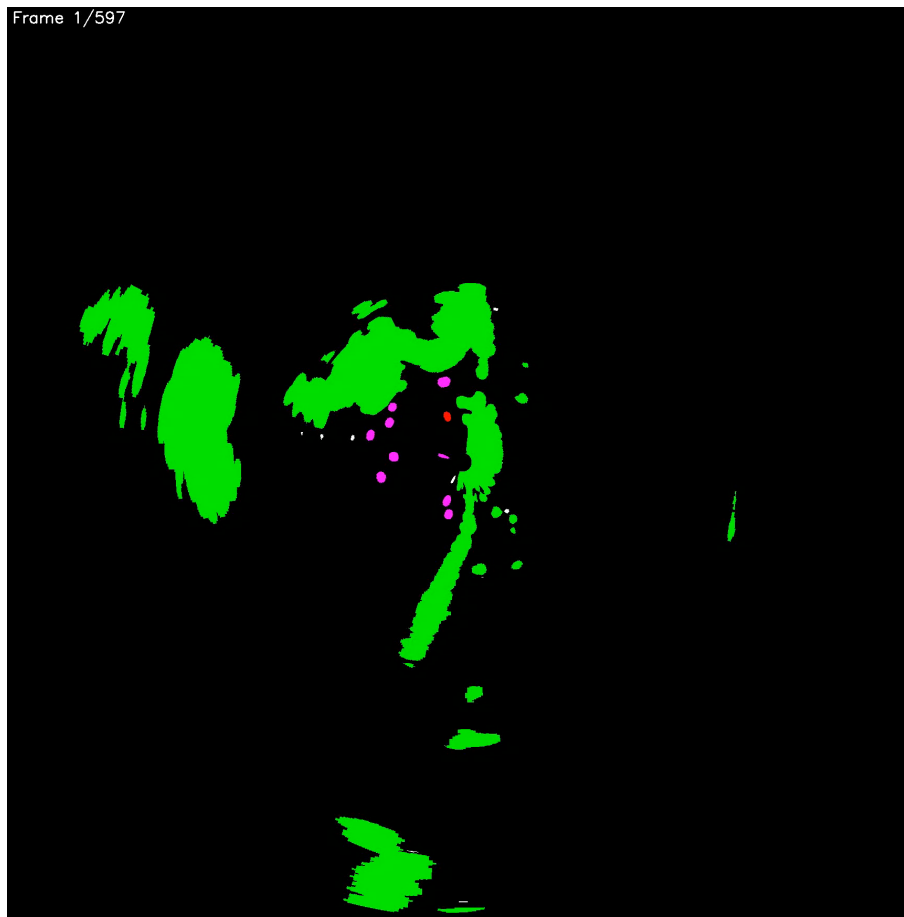
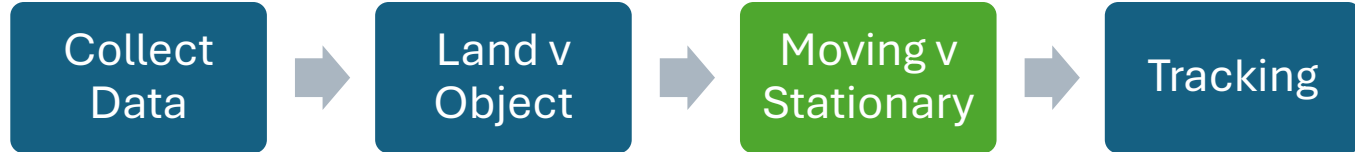
Current Pipeline



Current Pipeline



Current Pipeline



Current Pipeline

Collect
Data



Land v
Object



Moving v
Stationary



Tracking

Frame 1/597
Tracks present: 13



10-16-25 Architecture Overview

The Four main paths

J.C. Vaught

Undergraduate Assistants: Samuel Cancilla and Nathan Kirk
Department of Mechanical Engineering

Four Pipeline Architectures

1 STAGE

Single-Stage

Video → Objects + Land + Velocity/IDs in one pass. Joint detection and tracking with models like CenterTrack and TransTrack.

2 STAGES

Two-Stage

Detector/Classifier → Tracker. Get boxes and classes first, then compute identity and velocity with SORT-family trackers.

3 STAGES

Three-Stage

Detector → Classifier → Tracker. Split object detection from classification for cheaper detection and stronger few-shot classifiers.

4 STAGES

Four-Stage

Land Discriminator → Blob Detection → Classifier → Tracker. Explicitly strip out static land first, then process sea-only blobs.

Single-Stage Architecture

This is kinda the end-goal of what I would like to eventually have.

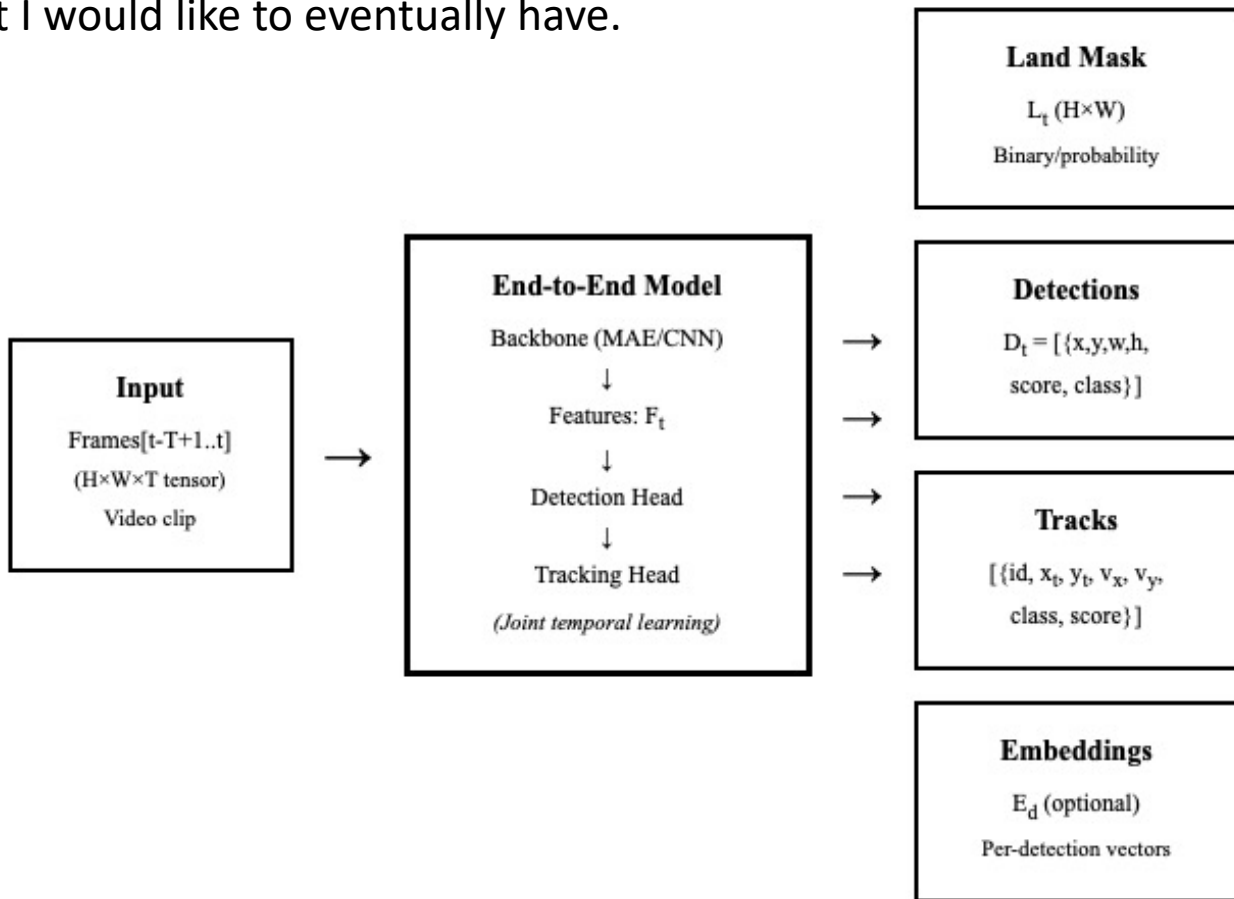
CenterTrack: 67.3% MOTA @ 22 FPS on MOT17, 89.4% MOTA on KITTI - online & real-time

TransTrack: End-to-end MOT with unified detection+association using Transformer queries

TrackFormer: State-of-the-art on MOT17/MOT20 at publication (CVPR 2022)

TransVOD++: 90.0% mAP on ImageNet-VID, ~30 FPS @ 83.7% mAP (Lite version)

CenterNet: 28.1% AP @ 142 FPS on COCO with multi-task heads



Two-Stage Architecture

Detectors

YOLOv7-Ship: +2.2% mAP vs YOLOv7-Tiny @ ~75 FPS, better small ships

YOLOv7-Sea: Improved tiny-object head for maritime UAV (WACV'23)

Seq-NMS: Temporal boost via rescoring across frames

FGFA: Won ImageNet-VID 2017, optical flow feature aggregation

Trackers

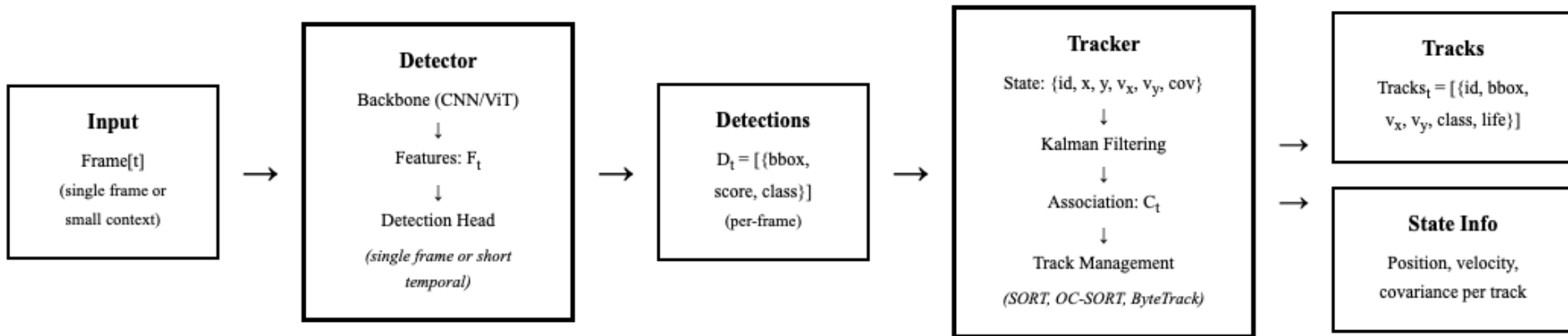
SORT: Very fast motion-only, ~260 Hz, Kalman Filter + IoU

DeepSORT: Adds appearance embeddings, better ID stability

ByteTrack: 80.3 MOTA / 77.3 IDF1 @ 30 FPS on MOT17

OC-SORT: 700+ FPS on CPU, SOTA metrics, robust in occlusion

BoT-SORT: 80.5 MOTA / 80.2 IDF1 / 65.0 HOTA on MOT17



Three-Stage Architecture

Detectors

CFAR: CA-CFAR, OS-CFAR, GO/SO on radar - classical, robust, parameter-light

Thresholding: Simple global/percentile threshold + morphology for cheap proposals

CNN Detector: YOLO/CenterNet for better recall on small objects

Classifiers

MAE Linear: Self-supervised ViT features + linear probe, strong transfer with few labels

Prototypical Nets: Few-shot metric learning, works with very few examples per class

SVM/Logistic: Hand-crafted blob features (area, eccentricity, intensity stats)

Small CNN: MobileNetV3 head with transfer learning

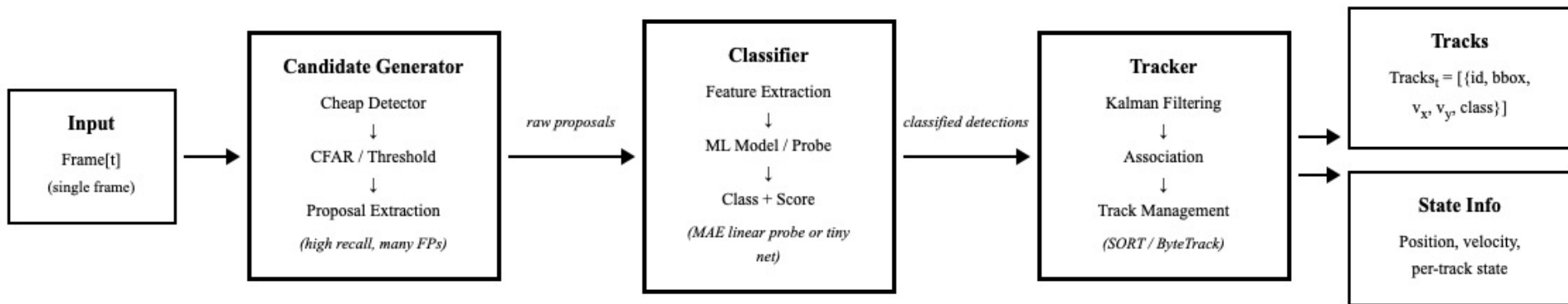
Trackers

Kalman + GNN: Provides position & velocity, simplest approach

JPDA: Joint Probabilistic Data Association for ambiguous assignments

RFS Filters: PHD / LMB / GLMB for principled multi-target with birth/death

MOT Trackers: SORT/DeepSORT/ByteTrack/OC-SORT/BoT-SORT



Four-Stage Architecture

Land Masking

Temporal Median: Background modeling for quasi-stationary platforms
GMM/MoG: Gaussian Mixture Models (Stauffer-Grimson), classic & fast
Radar Segmentation: X-band radar coastline extraction via DeepLabv3+

Classifiers

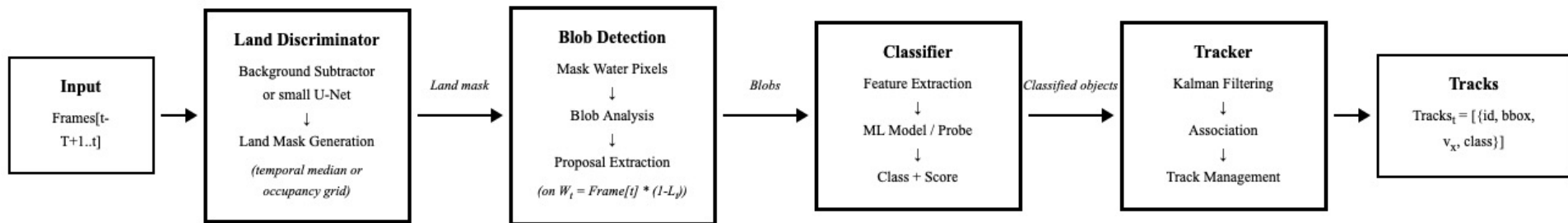
MAE Linear: Best accuracy-to-labels ratio (~20 frames)
ProtoNets: Few-shot episodic training for support/query sets
Tiny CNN: Train only last layers with heavy augmentation

Blob Detection

CCL: Connected Components Labeling - two-pass, union-find, GPU implementations
Morphology: Remove speckles, keep buoy/ship-scale blobs
Size Filters: Area, eccentricity, extent features

Trackers

Kalman/GNN: Land masking reduces false positives
ByteTrack: Handles misses gracefully with low-score detection association
OC-SORT: CPU-fast, robust during occlusion
JPDA/RFS: For frequent dropouts and missed detections



Questions?

J.C. Vaught

Department of Mechanical Engineering
University of South Carolina